

# Paper Critique

*By Bhuwan Pandey*

## ***Reasoning Analytically About Password-Cracking Software***

Enze Liu, Amanda Nakanishi, Maximilian Golla, David Cash, Blase Ur

*IEEE Symposium on Security and Privacy, 2019*

### **1. Summary**

There is a well-known disconnect in password security research: the academic community has largely built its threat models around probabilistic cracking algorithms such as Markov models, PCFGs, neural networks, while real world attackers predominantly rely on transformation based tools such as John the Ripper (JtR) and Hashcat. The reasons are pragmatic. Probabilistic models carry a non-trivial per-guess generation cost, which becomes a serious bottleneck when attacking fast hash functions at GPU speeds. JtR and Hashcat sidestep this by applying handcrafted mangling rules to large wordlists, trading theoretical elegance for sheer throughput. Liu et al. confront this gap directly, introducing the first computationally efficient framework for reasoning analytically about mangled wordlist attacks, without actually running them.

The central challenge the paper addresses is scale. The SpiderLabs rule list paired with the 60 million entry LinkedIn wordlist produces roughly  $3 \times 10^{14}$  guesses. Enumerating and storing all of them would take an estimated 4.7 years of compute time and approximately 3.3 petabytes of disk space. The authors' solution is to develop two algorithmic primitives that bypass enumeration entirely.

The first primitive, rule inversion, takes a rule and a target password as input and returns a compact regular expression describing every wordlist entry that the rule would transform into that password. For example, to determine whether the rule "append two digits" would ever produce password156, the tool works backwards, strip the two trailing digits, obtain password1, and reduces the problem to a single hash table lookup against the wordlist. For the large majority of transformations in both JtR and Hashcat, the resulting regular expression is finite and small, making the lookup extremely fast regardless of wordlist size.

The second primitive, guess counting, determines how many total guesses a given rule will produce on a given wordlist without executing the software. This is considerably more complex than inversion, particularly because rejection transformations rules that conditionally discard guesses based on properties like length or character class membership, interact with other transformations in non-trivial ways. The authors handle this through a precomputation scheme: they extract abstract features of the wordlist (word-length distributions, character-class membership counts), encode them in multidimensional lookup tables, and reduce each rule's guess count to a near-constant-time table query. Precomputation is a one-time cost of up to 16 hours for the largest wordlist tested; after that, each password lookup takes under one second on average.

## 2. Background and Motivation

The motivation for this work sits at the intersection of two well documented problems. The first is the practical dominance of transformation-based tools. As Goodin (2013) and Ur et al. (2015) documented, experienced attackers use JtR and Hashcat not out of habit but because they are genuinely faster for the most common attack scenario: cracking large sets of hashed passwords from data breaches using GPU hardware. For fast hash functions such as MD5, probabilistic models impose a generation overhead that is difficult to justify when the bottleneck is hash throughput, not candidate quality.

The second problem is that the academic community's standard configurations of these tools significantly underestimate password vulnerability. Ur et al. (2015) showed this empirically, but the demonstration itself required exhaustive enumeration, the very bottleneck that Liu et al. set out to eliminate. Without an efficient way to reason about JtR and Hashcat, researchers could not systematically explore the configuration space, could not deploy these models for realtime password checking, and could not make principled comparisons between rule lists or wordlists. The paper fills that gap.

## 3. Core Technical Contributions

The inversion algorithm is the technical heart of the paper. The key observation is that most individual transformations in JtR's and Hashcat's rule languages are finite-to-one functions. i.e. for a given output, the set of inputs that produce it is finite and structurally computable. The substitution rule  $sXY$ , for instance, has a systematic inverse, replace every occurrence of  $Y$  in the target password with the character class  $[XY]$ , further yielding a simple regular expression whose members are exactly the preimages.

Handling composed rules is where the approach becomes non-trivial. The authors show that if each individual transformation in a composition is fully invertible, the composition can be inverted by chaining the inversion steps in reverse order, feeding the output regular expression of one step as input to the previous. Crucially, they prove that compositions of fully invertible transformations always yield a simple yet finite regular expression, and hence a result that underpins the efficiency of the system. The two exceptions are truncation and substring extraction, which yield not simple (potentially infinite) regular expressions and require a linear scan of the wordlist rather than enumeration. The six not invertible transformations, which are two character purging commands and four memory commands, are handled by falling back to direct execution of JtR or Hashcat. Character purging fails because its inversion under composition requires exponentially branching case analysis; memory commands fail because they are Turing-complete in the general case.

## 4. Related Work

The paper relates to two prior streams of work. The first is the literature on efficient guess number calculation for probabilistic password models. Dell'Amico and Filippone's Monte Carlo method enables efficient estimation for Markov models and PCFGs; Melicher et al.'s compact neural network architecture enables client-side estimation that runs in a browser. The authors correctly note that none of these techniques transfer to transformation based tools, which do not assign probabilities to passwords and therefore offer no probability distribution to sample from.

The second stream is the prior work on configuring and analyzing JtR and Hashcat. Marechal (2012) proposed ordering rules by empirical success rate and generating prepend/append rules from evaluation data; Kacherginsky (2013) used reverse Levenshtein paths to generate rules automatically. Both approaches required full enumeration of guesses and could not scale to the rule list and wordlist sizes used in practice. Chrysanthou (2013) developed a sophisticated multistage attack configuration through extensive empirical experiments, again limited to a single test set and dependent on full execution. Liu et al. provide the analytical foundation that makes all of these directions rigorous and scalable. A connection worth noting even though the authors touch on it only lightly has become the structural similarity between the backwards constraint propagation in guess counting and weakest precondition computation in formal program verification. This however, readers from a programming-languages background will find thread interesting.

## 5. Assumptions, Design Choices, and Experimental Results

The paper explicitly scopes itself to trawling attacks. i.e. non-targeted, large-scale offline cracking, and this boundary is clearly maintained throughout. Targeted attacks, which leverage victim specific information or credential reuse from prior breaches, involve fundamentally different wordlist and rule configurations and are out of scope. This is a reasonable constraint for a first paper of this kind, though it is worth keeping in mind when interpreting the practical implications.

The performance results are the most immediately compelling part of the evaluation. For the SpiderLabs + LinkedIn configuration, the analytical approach reduces the per password lookup to a mean of 0.367 seconds after a 16 hour one-time precomputation, with auxiliary storage of 10.2 GB as compared to an estimated 4.7 years of enumeration time and 3.3 petabytes of storage. Across the six rule list and wordlist combinations tested, the pattern holds large, highly invertible rule lists paired with large wordlists benefit the most, while configurations with higher proportions of uninvertible rules see more modest but still substantial improvements.

The wordlist reordering negative result is one of the paper's more intellectually honest contributions. The expectation that data driven reordering of wordlists for Hashcat would mirror the gains seen with rule reordering for JtR turned out to be wrong. Meaning, reordering consistently degraded performance on held out evaluation sets. This finding is reported

prominently and discussed thoughtfully, which is not always the case for negative results in systems security papers.

The rule completeness experiments demonstrate that the existing publicly available rule lists leave significant configuration space unexplored. Extending SpiderLabs with the 15,085 single transformation JtR rules and then reordering the extended list resulted in both faster cracking and coverage of passwords that the original configuration would never have guessed. Of the 3,495 newly executed rules, 3,202 were entirely absent from all three JtR lists tested. The wordlist completeness results are similarly revealing. i.e. the top missing preimages identified via rule inversion include site-specific words, semantically meaningful strings, and short character combinations, all classes of entries that are difficult to recover by other means.

The comparison against existing password meters in Section XII is carefully designed. The authors follow Golla and Durmüth's recommended methodology of computing the weighted Spearman correlation between a meter's guess numbers and ground truth password frequency counts. The JtR and Hashcat analytical approaches outperform zxcvbn, neural networks, Markov models, and PCFGs on this metric across all six evaluation sets. The analytical approach only assigns guess numbers to passwords it can crack (around 40–80%), unlike probabilistic models that cover all passwords. This is fine for server-side checkers but limits effectiveness for client-side meters that must evaluate every input.

## **6. Key Strengths**

The primary strength of this paper is that its core contribution is both technically novel and rigorously validated. The inversion and guess counting algorithms required a thorough formal analysis of two large, underdocumented rule languages, and the validation methodology — 100,000 randomly generated composed rules per tool, compared exhaustively against ground truth, sets a high standard. The result is a system that can be trusted to do what it claims, which is not a given for research prototypes in this space.

The evaluation is notably broad and honest. Six evaluation sets, three wordlists, six rule lists, and four categories of prior meters give the results real weight. The inclusion of a non-English evaluation set (CSDN), the transparent reporting of precomputation costs, the quantification of uninvertible rule proportions, and most importantly, the prominent reporting of the wordlist reordering negative result all contribute to a paper that earns credibility rather than just asserting it.

The practical relevance of the applications is also a genuine strength. The guess number calculator directly enables server-side password checking grounded in actual attacker behavior something which no prior meter could offer. The opensource release of all tools means the community can build on this work rather than having to reconstruct it.

## 7. Key Weaknesses

The most significant technical limitation is the uninvertible transformation coverage gap. Character purging rules and memory commands affect up to 21% of Hashcat's Generated2 list and fall back to bruteforce execution. For expert curated non-public lists, i.e. the ones most likely to use complex and aggressive transformations, the proportion of uninvertible rules could be higher, but the paper cannot assess this because those lists are not publicly available. A more detailed theoretical analysis of what fraction of "realworld" rules are likely to be uninvertible would strengthen the paper's claims about practical applicability.

The precomputation scalability story is incomplete. The Generated2 + LinkedIn configuration requires 9.8 TB of auxiliary data, which is significant even by research standards, and the paper does not address incremental updates. In a production deployment where the wordlist is periodically refreshed with new breach data, recomputing from scratch each time would be costly. This gap between the research contribution and an operational tool is worth acknowledging more directly.

Finally, the scope limitation to trawling attacks, while clearly stated, means the paper's results do not directly address a significant fraction of high-value real-world compromises. Targeted attacks involving personal information or credential reuse may require fundamentally different configurations, and it is unclear how the optimization techniques developed here would generalize to those settings.

## 8. Conclusion and Future Work

Liu et al. deliver a contribution that is simultaneously technically solid and practically consequential. The analytical framework they introduce does not merely improve on existing methods but enables a category of research that was previously inaccessible due to computational constraints. The guess-number calculator is immediately deployable for server-side password strength estimation; the optimization techniques give researchers a principled approach to configuration that did not exist before; and the open-source release ensures the community can build on rather than duplicate this work.

The authors identify several natural directions for future work: extending inversion to handle memory commands for common special cases, developing incremental update strategies for the precomputed auxiliary structures, and studying how users adapt their password choices in response to a JtR/Hashcat-based checker. Perhaps the most consequential open direction, however, is the one the paper implicitly raises but cannot address: applying these techniques to the non-public expert-curated lists that experienced attackers actually use. The paper demonstrates convincingly that public configurations are suboptimal, characterizing what optimal looks like either through access to expert lists or through principled synthesis, seems like the natural and most impactful next step for this line of research.